

Floating-Point Addition

Computer Architecture & Organization Notes

June 27, 2025

1 Introduction to Floating-Point Addition

Floating-point addition is one of the fundamental operations in computer arithmetic. Unlike integer addition, floating-point addition requires several steps to ensure proper alignment, normalization, and rounding. This document provides a comprehensive explanation of the floating-point addition process with detailed examples.

1.1 Floating-Point Representation

A floating-point number is typically represented as:

$$\pm \text{significand} \times \text{base}^{\text{exponent}} \quad (1)$$

For decimal representation: $\pm d_0.d_1d_2 \dots d_n \times 10^e$

For binary representation: $\pm 1.b_1b_2 \dots b_n \times 2^e$ (normalized form)

2 The Four-Step Process of Floating-Point Addition

Floating-point addition follows a systematic four-step process to ensure accuracy and proper representation of the result.

2.1 Step 1: Align the Decimal Points

Purpose: Before we can add two floating-point numbers, we must align their decimal points by making their exponents equal.

Process:

1. Compare the exponents of both numbers
2. Identify the number with the smaller exponent
3. Shift the significand of the smaller number to the right
4. Adjust the exponent to match the larger exponent

Important Note: Shifting right may cause loss of precision (digits are lost from the right side).

2.2 Step 2: Add the Significands

Purpose: Once the decimal points are aligned, we can perform standard addition on the significands.

Process:

1. Add the significands using standard arithmetic rules
2. Keep track of any carry that may occur
3. The result may not be in normalized form

2.3 Step 3: Normalize the Result

Purpose: Ensure the result is in proper normalized form and check for special conditions.

Normalization Rules:

- For decimal: The significand should be in the form $d.d_1d_2\dots$ where $1 \leq d \leq 9$
- For binary: The significand should be in the form $1.b_1b_2\dots$ (implicit leading 1)

Special Conditions to Check:

- **Overflow:** Exponent becomes too large to represent
- **Underflow:** Exponent becomes too small to represent
- **Zero result:** Handle as a special case

2.4 Step 4: Round the Result

Purpose: Adjust the result to fit within the available precision limits.

Rounding Rules (Round-to-Nearest):

- If the digit to be discarded is 0-4: Truncate (round down)
- If the digit to be discarded is 5-9: Round up (add 1 to the last kept digit)
- Handle carries that may result from rounding

3 Detailed Example: Decimal Floating-Point Addition

Let's work through the example: $9.999 \times 10^1 + 1.610 \times 10^{-1}$

Given Constraints:

- 4 decimal digits for significand
- 2 decimal digits for exponent

3.1 Step 1: Align Decimal Points

Initial numbers:

$$\text{Number 1: } 9.999 \times 10^1 = 99.99 \quad (2)$$

$$\text{Number 2: } 1.610 \times 10^{-1} = 0.1610 \quad (3)$$

Comparison of exponents:

- Exponent of Number 1: +1
- Exponent of Number 2: -1
- Difference: $1 - (-1) = 2$ positions

Alignment process: Number 2 has the smaller exponent, so we shift its significand right by 2 positions:

$$1.610 \times 10^{-1} \rightarrow 0.1610 \times 10^1 \quad (4)$$

$$\rightarrow 0.01610 \times 10^1 \quad (5)$$

After alignment:

$$\text{Number 1: } 9.999 \times 10^1 \quad (6)$$

$$\text{Number 2: } 0.01610 \times 10^1 \text{ (Note: precision lost)} \quad (7)$$

With 4-digit precision: Number 2 becomes 0.0161×10^1

3.2 Step 2: Add the Significands

$$9.999 + 0.0161 = 10.0151 \quad (8)$$

Result: 10.0151×10^1

3.3 Step 3: Normalize the Result

The result 10.0151×10^1 is not in normalized form because the significand is ≥ 10 .

Normalization:

$$10.0151 \times 10^1 = 1.00151 \times 10^2 \quad (9)$$

Check for overflow/underflow:

- Exponent is +2, which fits in 2 digits: OK
- No underflow or overflow detected

3.4 Step 4: Round the Result

With 4-digit significand precision, we need to round 1.00151:

- Digit to be discarded: 5 (5th digit)
- Since $5 \geq 5$, we round up

Rounding process:

$$1.00151 \rightarrow 1.002 \quad (10)$$

Final Result: $1.002 \times 10^2 = 100.2$

4 Binary Floating-Point Addition Example

Let's add $0.5 + (-0.4375)$ in binary using IEEE 754 single-precision format.

4.1 Step 1: Convert to Binary Floating-Point

Converting 0.5:

$$0.5_{10} = 0.1_2 = 1.0_2 \times 2^{-1} \quad (11)$$

Converting 0.4375:

$$0.4375_{10} = 0.0111_2 = 1.11_2 \times 2^{-2} \quad (12)$$

So we need to compute:

$$1.0_2 \times 2^{-1} + (-1.11_2 \times 2^{-2}) \quad (13)$$

4.2 Step 2: Align Exponents

Exponent comparison:

- Number 1 exponent: -1
- Number 2 exponent: -2
- Larger exponent: -1

Align Number 2 to exponent -1:

$$-1.11_2 \times 2^{-2} = -0.111_2 \times 2^{-1} \quad (14)$$

Aligned numbers:

$$\text{Number 1: } +1.000_2 \times 2^{-1} \quad (15)$$

$$\text{Number 2: } -0.111_2 \times 2^{-1} \quad (16)$$

4.3 Step 3: Add Significands

Since we have opposite signs, this becomes subtraction:

$$1.000_2 - 0.111_2 = 0.001_2 \quad (17)$$

Result: $0.001_2 \times 2^{-1}$

4.4 Step 4: Normalize

The result $0.001_2 \times 2^{-1}$ needs normalization:

$$0.001_2 \times 2^{-1} = 1.0_2 \times 2^{-4} \quad (18)$$

4.5 Step 5: Verify

Convert back to decimal:

$$1.0_2 \times 2^{-4} = 1 \times 2^{-4} = \frac{1}{16} = 0.0625_{10} \quad (19)$$

Check: $0.5 - 0.4375 = 0.0625$

5 Common Issues and Special Cases

5.1 Loss of Precision

- Occurs during alignment when significant digits are shifted out
- More severe when exponents differ greatly
- Can lead to catastrophic cancellation

5.2 Overflow and Underflow

- **Overflow:** Result too large to represent (exponent exceeds maximum)
- **Underflow:** Result too small to represent (exponent below minimum)
- Usually handled by setting to infinity or zero respectively

5.3 Catastrophic Cancellation

When subtracting nearly equal numbers, significant digits can be lost:

$$1.000000 - 0.999999 = 0.000001 \quad (20)$$

Most significant digits cancel, leaving only rounding errors.

6 Hardware Implementation Considerations

6.1 Alignment Shifter

- Barrel shifter for efficient multi-bit shifts
- Sticky bit to track lost precision

6.2 Addition/Subtraction Unit

- Standard binary adder/subtractor
- Extra guard bits for intermediate precision

6.3 Normalization Logic

- Leading zero detector
- Priority encoder for shift amount
- Exponent adjustment logic

6.4 Rounding Unit

- Implements various rounding modes
- Handles ties and special cases

7 Summary

Floating-point addition is a complex process involving four main steps:

1. **Alignment:** Match exponents by shifting significands
2. **Addition:** Add or subtract aligned significands
3. **Normalization:** Ensure proper format and check for exceptions
4. **Rounding:** Fit result within precision limits

Key considerations include:

- Precision loss during alignment
- Proper handling of special cases (overflow, underflow, zero)
- Accurate rounding to maintain precision
- Efficient hardware implementation

Understanding these concepts is crucial for computer architects and programmers working with floating-point arithmetic, as it affects both performance and numerical accuracy of computations.